

Deep Learning with Medical Imaging

Report

Vinson Cai

Executive Summary

I undertook a Deep Learning Project where I chose to analyse a dataset of brain MRI images. This stemmed from a personal desire to explore the potential role of deep learning being used to effectively diagnose patients and perform research in the medical field in the near future.

This report details the application of various methods associated with deep learning, specifically with neural networks. The stages in this project are as follows:

1. Problem Specification
2. Collection and Pre-processing of Data
3. Exploratory Data Analysis (EDA)
4. Benchmark Model (Non-Deep Learning)
5. Deep Learning Model Architectures
6. Hyperparameter Tuning
7. Discussion of Results
8. Conclusion

1 Problem Specification

The supervised learning problem chosen is to accurately predict whether or not a given patient has a particular type of brain tumor or does not have a brain tumor, based on a given MRI image of the brain. This is a **multi-classification** problem with four possible categories: glioma (3), meningioma (2), pituitary tumor (1) and no tumor (0).

2 Collection of Data

The dataset, sourced from the site Kaggle, contains 7023 public domain MRI images of the brain at various angles. It was published and last updated in 2021, by a user named Masoud Nickparvar.

3 Exploratory Data Analysis (EDA) and Pre-processing

Several insights were gained through carrying out an EDA of the dataset. Some details found warranted multiple pre-processing steps prior to inputting the data into a machine learning model.

Firstly, images in the dataset vary in pixel size and was a combination of grayscale and RGB (colour) images as shown in Figure 3 (Appendix). Pre-processing is required to tackle dataset inconsistency, reduce the computational cost of training for machine learning models and simplify the data whilst maintaining the complexity and detail of all images. The pre-processing involved converting each image to greyscale and resizing it to a 128 x 128 matrix, with each value representing pixel intensity. Those values were

normalised to be between 0 and 1 inclusive instead of an integer between 0 and 255.

Additional pre-processing steps that all images undergo to improve the performance of machine learning models is noise reduction, contrast enhancement and edge preservation. This may improve the ability of a model to detect potential brain tumour boundaries.

The dataset is relatively balanced as shown in Table 4 (Appendix). Due to the relatively small number of observations provided, a conscious decision was made to refrain from downsampling the data to an even 25-25-25-25 split in the categories.

Additionally, the distribution of pixel intensity values (after normalisation) for singular images (Figure 4(a) and 4(b) in Appendix) and the entire dataset was analysed (Figure 4(c) in Appendix). Models may recognise the prevalence of a certain pixel value as a strong feature, which convolutional neural networks can use to differentiate between images of different categories. A strong feature could correspond to a brain tumor boundary or certain types of brain tissue.

To make comparing outputs more machine learning friendly, each category in the response was encoded as integers from 0 to 3. The data was then randomly split 60-20-20 into 3 sets. This represents the train, validation and test set respectively. If the validation set is unnecessary, the training data is the first random 80% and the test set is other 20%. The data was also reshaped accordingly for each machine learning model.

4 Non-Deep Learning Benchmark Model

A simple decision tree was fitted to the dataset as the non-deep learning benchmark model. This model will be used to assess various deep learning architectures.

The type of model used was a simple decision tree. With the model training on 60% of the data, the test accuracy on the 20% of the data was 0.8484 and the F1-score was 0.8471.

With an 80-20 train-test split, the test accuracy and F1-score improved to 0.8726 and 0.8724 respectively.

5 Deep Learning Model Architectures

5.1 Introduction

The neural network is the most prevalent type of deep learning model. A specific type of neural network in the realm of computer vision that is designed to effectively train on image datasets is the convolutional neural network (CNN). We use a CNN instead of the classic feedforward artificial neural network (ANN) as the input layer becomes extremely large with images (recall each image is a matrix of 128 x 128 values).

The 2 main types of layers that characterise the CNN is the convolutional layer and the pooling layer. The convolutional layer extracts the features of the image by sliding the filter kernels that are of a certain dimension over parts of the matrix and calculating the dot product after every stride. The pooling layer downsamples the output of a convolution layer by taking a max or average value out of a certain size of the image.

A pooling layer typically directly follows a convolutional layer in an architecture of a CNN. Repetitions of this structure, before flattening the data, followed by an output layer are typical of a CNN. Fully connected layers can also be added after flattening the data.

For my supervised learning problem, I constructed two different CNNs, which were both hyperparameter tuned manually and with keras-tuner and fitted accordingly. A 60-20-20 train-validation-test split was

used. Both use the Keras Sequential Application Programming Interface. The advantage of the Sequential API allows for a user to build the layers one by one and in a desired order.

The following describes the process of building the architectures of both models in separate sections, the justifications for major decisions during this process and the initial results.

5.2 Model 1

The first deep learning model is a simple CNN model which trains in a relatively short amount of time. It is slightly more computationally expensive than the benchmark model. The simplicity of this model allows for appropriate comparison against more complex neural networks. This model has 142,484 parameters, all of which are trainable. Further details of the structure and performance of this model is in Table 2 Model 1 and the Discussion of Results.

5.3 Model 2

This second deep learning model is a more complex CNN model that is significantly more computationally expensive than Model 1 and trains for a greater amount of time. This model has a slightly greater test accuracy due to this model being more polished. This model has 10,291,504 parameters, all of which are trainable. Further details of the structure and performance of this model is in Table 2 Model 2 and the Discussion of Results.

6 Hyperparameter Tuning

Most of the hyperparameter tuning was performed during the deliberation of constructing and selecting the best parameters for the simple CNN (Model 1). Table 2 above shows the structure and test accuracy for a subset of models that were trained manually.

Tuning variables on simpler models gave some insights into the optimal setting of parameters. Below are some discoveries made throughout the manual hyperparameter tuning process:

- Max pooling was used over average pooling as it picks up on strong, prominent features better.
- The model performs better with the Leaky ReLU activation function. The model performed comparatively worse with the ReLU or tanh activation functions.
- Padding improves the accuracy of the model.
- The stride and pooling size should not be more than 3 x 3 to maximise the accuracy.
- If dense layers were used, adding a dropout layer often slightly improved the performance of the model.

Hyperparameter tuning helped in assist in choosing the 2 most appropriate deep learning models for this report. Some consistent parameters used throughout all models:

- The output layer was always a Dense layer of 4 neurons with the softmax activation function. Softmax is chosen as it is the most appropriate for multi-classification.
- The loss function for which the models were compiled was always sparse categorical cross entropy. Note that "sparse" is used as the outputs are integers and not vectors.
- The metric was sparse categorical accuracy.
- The optimiser was Adam.
- Early stopping with a patience of 5 or 10 epochs was used for all models to prevent the model overfitting to the training data. The model weights in the final model were restored from five less than the number of epochs trained for.

Table 1: Results from Manual Hyperparameter Tuning

Model No.	No. Conv. + Pool. Layers	No. Dense Layers	No. Neurons per Layer	Activation Functions	Kernel Size	Pooling Size	Stride Size	Pooling Type	Padding Used? / Dropout Used?	No. Epochs	Test Accuracy
1	1	0	80, F, 4	Leaky ReLU, Softmax	4 x 4	2 x 2	3 x 3	Max	True / False	21	0.9352
2	2	3	50, 50, F, 200, 200, 4	Leaky ReLU x 4, Softmax	2 x 2	2 x 2	1 x 1	Max	True / True	22	0.9530
3	1	1	100, F, 4	Leaky ReLU, Softmax	4 x 4	4 x 4	1 x 1	Average	False / False	20	0.9253
4	1	1	100, F, 4	Leaky ReLU, Softmax	4 x 4	4 x 4	2 x 2	Max	False / False	34	0.9153
5	1	1	100, F, 4	Leaky ReLU, Softmax	4 x 4	4 x 4	1 x 1	Max	False / False	28	0.9367
6	1	1	500, F, 4	Leaky ReLU, Softmax	2 x 2	2 x 2	1 x 1	Max	True / False	14	0.9302
7	1	1	500, F, 4	Leaky ReLU, Softmax	3 x 3	3 x 3	1 x 1	Max	True / False	17	0.9381
8	1	1	100, F, 4	Leaky ReLU, Softmax	3 x 3	3 x 3	1 x 1	Max	True / False	20	0.9345
9	2	1	100, 100, F, 4	Leaky ReLU x 2, Softmax	3 x 3	3 x 3	1 x 1	Max	True / False	20	0.9324
10	2	1	100, F, 4	ReLU x 2, Softmax	4 x 4	4 x 4	1 x 1	Average	False / False	24	0.9274
11	4	1	50, 100, 150, 200, F, 4	Leaky ReLU x 4, Softmax	2 x 2	2 x 2	2 x 2	Max	True / False	20	0.9445
12	7	1	10, 25, 50, 100, 50, 25, 10, F, 4	Leaky ReLU x 7, Softmax	2 x 2	2 x 2	2 x 2	Max	True / False	25	0.9196
13	7	1	10, 25, 50, 100, 50, 25, 10, F, 4	Leaky ReLU x 7, Softmax	2 x 2	2 x 2	2 x 2	Average	True / False	24	0.8577
14	7	1	20, 20, 20, 20, 20, 20, 20, F, 4	Leaky ReLU x 7, Softmax	2 x 2	2 x 2	2 x 2	Max	True / False	34	0.9146
15	0	4	F, 100, 100, 100, 4	Leaky ReLU x 4, Softmax	N/A	N/A	N/A	Max	False / False	15	0.8911
16	0	3	F, 100, 100, 4	Leaky ReLU x 7, Softmax	N/A	N/A	N/A	Max	False / True	33	0.9053
17	3	3	50, 50, 50, F, 200, 200, 4	Leaky ReLU x 6, Softmax	2 x 2	2 x 2	1 x 1	Max	True / True	15	0.9523
18	4	3	50, 50, 50, 20, F, 200, 200, 4	Leaky ReLU x 6, Softmax	2 x 2	2 x 2	1 x 1	Max	True / True	17	0.9260
19	1	1	100, F, 4	Leaky ReLU, Softmax	4 x 4	4 x 4	2 x 2	Average	False / False	42	0.9153
20	3	3	25, 50, 100, F, 200, 200, 4	Tanh x 5, Softmax	2 x 2	2 x 2	1 x 1	Max	True / True	13	0.9359

7 Discussion of Results

The performance of each model was evaluated with respect to the accuracy of prediction on the unseen test set. Both CNN models performed considerably better than the benchmark model. Model 2, the more complex model, performed slightly better than model 1.

It is observed in Figure 1 that training and validation accuracy approaches 1 more quickly for Model 2 compared to Model 1. In Figure 2, it is observed that the training loss approaches 0 much faster for Model 2 than in Model 1. Validation accuracy and loss in the later epochs is fairly constant though Model 2 tends to have more variance than Model 1 in both graphs. All this makes sense as the complex model (Model 2) is expected to learn at a much faster rate than the simpler model (Model 1).

Both models follow the trend of predicting some categories better than others. Both predict images with no tumor and pituitary tumors very accurately. Glioma is predicted accurately at a worse rate and meningioma is predicted correctly at the poorest rate. This is evident by the F1-Scores for each category

(shown in Table 2 and Table 3). To improve both models, either the precision and recall scores for a certain category must increase without lowering the other. For example, an improved model should be able to better identify meningioma (higher recall) whilst not trading off any other metrics. Note that in the context of identifying serious conditions, recall is slightly more important than precision.

Figure 1: Training and Validation Accuracy Over Approximately 20 Epochs

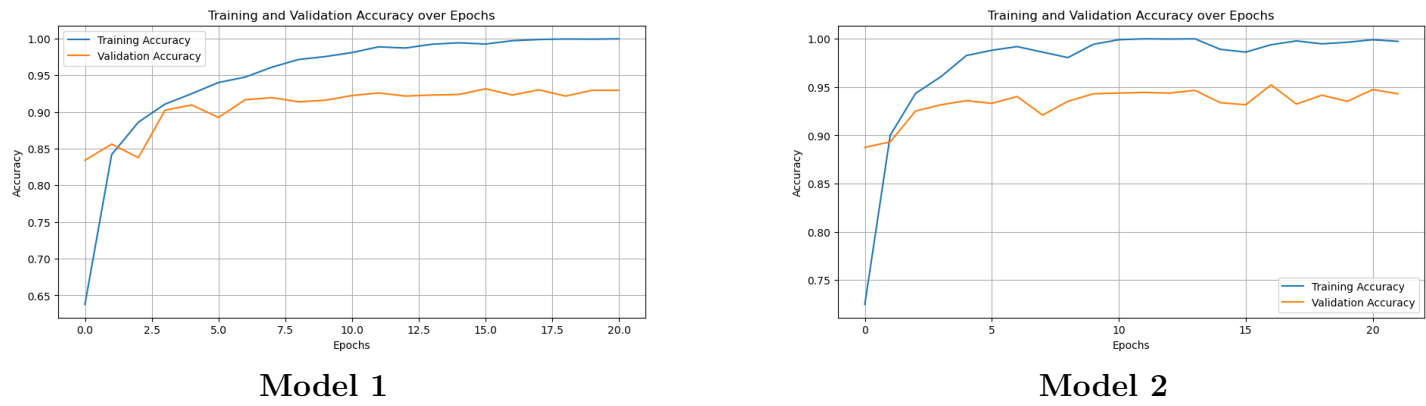


Figure 2: Training and Validation Loss Over Approximately 20 Epochs

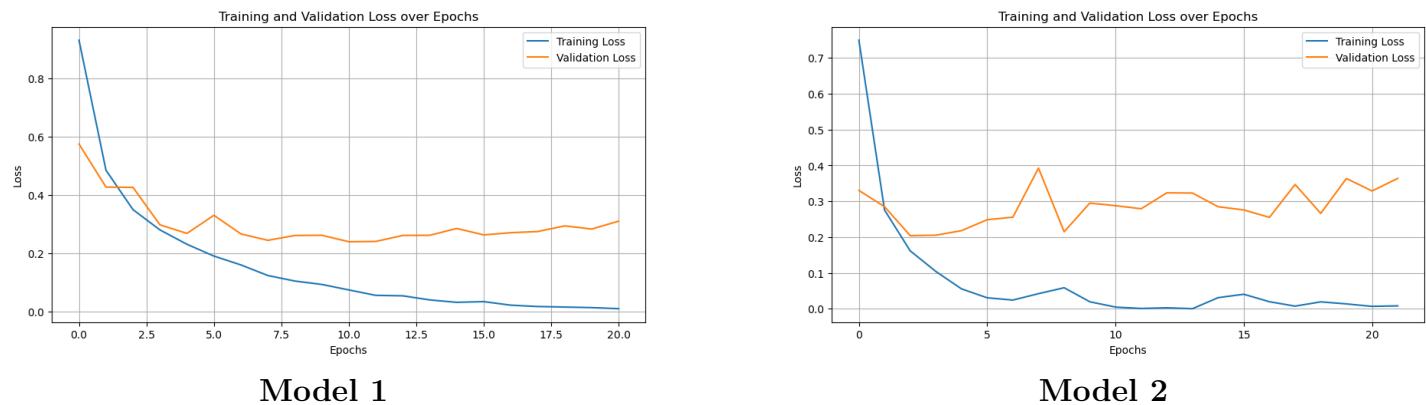


Table 2: Classification Metrics for Model 1

Class	Precision	Recall	F1-Score
No tumor (0)	0.96	0.99	0.97
Pituitary (1)	0.93	0.99	0.96
Meningioma (2)	0.90	0.85	0.87
Glioma (3)	0.95	0.91	0.93
Weighted Avg	0.93	0.94	0.93
Accuracy	0.9352 (1405 samples)		

Table 3: Classification Metrics for Model 2

Class	Precision	Recall	F1-Score
No tumor (0)	0.98	0.99	0.98
Pituitary (1)	0.95	1.00	0.97
Meningioma (2)	0.93	0.88	0.90
Glioma (3)	0.94	0.94	0.94
Weighted Avg	0.95	0.95	0.95
Accuracy	0.9523 (1405 samples)		

8 Summary and Conclusion

In this report, the performance of 2 CNN deep learning models were compared against each other and against a non-deep learning baseline model. Exploration of the different traits of CNNs was performed through the activity of hyperparameter tuning to select the ideal values for certain parameters for the CNN models. It was found that the best deep learning models were accurate around 95% of the time. The models presented here do not have a high enough recall and accuracy rate. If there are continual advancements in deep learning, it could be viable in serving a prominent role in the medical industry in the near future.

9 APPENDIX

9.1 Generative AI Usage

The only generative AI used throughout my project was ChatGPT. I have used it to assist me in several activities throughout this Deep Learning Project. Other generative AI such as Copilot was not used.

ChatGPT was used, for the most part, to assist and speed up the debugging process of pre-processing data, coding model architectures and performing hyperparameter tuning. Errors as well as the offending code would be used as the prompt. Suggestions and fixes given by ChatGPT would often be applied to the code. Debugging was not always successful.

Additional uses included exploring, finding and explaining the arguments of certain functions and libraries in Python.

This report was written with LaTeX on Overleaf. I used ChatGPT to efficiently find useful packages that were especially helpful in the tabulating of data and formatting of images and tables.

9.2 EDA

The following shows the disparity in image size and number of channels (1 for greyscale / 3 for RGB) between the images in the dataset.

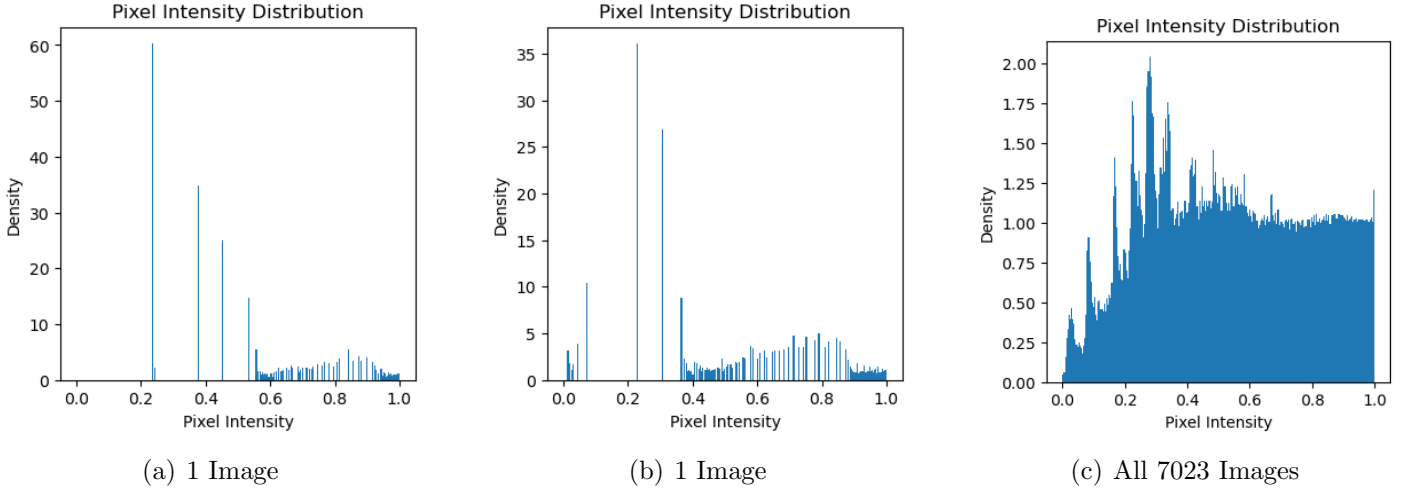
Figure 3: Image Type and Image Size for the 3754th to 3786th Image Processed

3754	('R', 'G', 'B') (512, 512)
3755	('R', 'G', 'B') (512, 512)
3756	('L',) (512, 512)
3757	('L',) (512, 512)
3758	('L',) (512, 512)
3759	('R', 'G', 'B') (201, 251)
3760	('R', 'G', 'B') (528, 581)
3761	('R', 'G', 'B') (213, 236)
3762	('R', 'G', 'B') (209, 241)
3763	('R', 'G', 'B') (341, 372)
3764	('R', 'G', 'B') (1149, 1019)
3765	('L',) (512, 512)
3766	('L',) (512, 512)
3767	('L',) (512, 512)
3768	('L',) (512, 512)
3769	('L',) (512, 512)
3770	('L',) (512, 512)
3771	('R', 'G', 'B') (722, 690)
3772	('R', 'G', 'B') (341, 395)
3773	('R', 'G', 'B') (200, 235)
3774	('R', 'G', 'B') (295, 394)
3775	('R', 'G', 'B') (224, 216)
3776	('R', 'G', 'B') (214, 235)
3777	('L',) (512, 512)
3778	('R', 'G', 'B') (225, 225)
3779	('R', 'G', 'B') (262, 308)
3780	('R', 'G', 'B') (214, 216)
3781	('R', 'G', 'B') (512, 512)
3782	('R', 'G', 'B') (206, 245)
3783	('R', 'G', 'B') (341, 395)
3784	('R', 'G', 'B') (314, 195)
3785	('L',) (512, 512)
3786	('L',) (512, 512)

Table 4: Counts and Percentages of Categories

Classification	Count	Percentage
No Tumor	2000	28.48%
Pituitary	1757	25.02%
Meningioma	1645	23.42%
Glioma	1621	23.08%
TOTAL	7023	100.00%

Figure 4: Pixel Intensity Distributions for a Random Sample of...



9.3 References

Dataset can be downloaded from:

<https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset>

- dshahid380 (2019). *Convolutional Neural Network*. [online] Medium. Available at: <https://towardsdatascience.com/covolutional-neural-network-cb0883dd6529#:~:text=Examples%20of%20CNN%20in%20computer.>